Лабораторная работа №1

Графические интерфейсы

Цель работы: научиться разрабатывать приложения, обладающие графическим интерфейсом пользователя, с использованием библиотеки Swing.

Теоретический материал

Сведения о библиотеке Swing и проектировании с ее использованием графических интерфейсов в NetBeans представлены в отдельном файле NetBeans_SWING.pdf.

Задание на лабораторную работу

Разработать приложение, обладающее графическим интерфейсом с использованием языка Java и библиотеки Swing. Приложение должно содержать 3 поля ввода (JTextField) доступных для редактирования и соответственно таблицу (JTable) с тремя колонками: нижняя граница интегрирования, верхняя граница интегрирования и шаг интегрирования. Кроме того, должны присутствовать 3 конопки (JButton): добавить, удалить, вычислить. Для добавления/удаления строки и вычисления значения определенного интеграла для функции в соответствии с вариантом задания (Приложение 1) и параметров выделенной строки таблицы. Результат должен выводиться в четвертое поле, которое не доступно на редактирование. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Лабораторная работа №2

Работа с коллекциями объектов

Цель работы: изучить библиотеку стандартных коллекций Java Collections Framework, позволяющую хранить различные структуры данных.

Теоретический материал

При написании программы очень часто возникает потребность хранить набор каких-либо объектов. Это могут быть числа, строки, объекты пользовательских классов и т.п. Для этих целей служит стандартная библиотека коллекций Java Collections Framework.

Базовые интерфейсы

В библиотеке коллекций Java существует два базовых интерфейса, реализации которых и представляют совокупность всех классов коллекций:

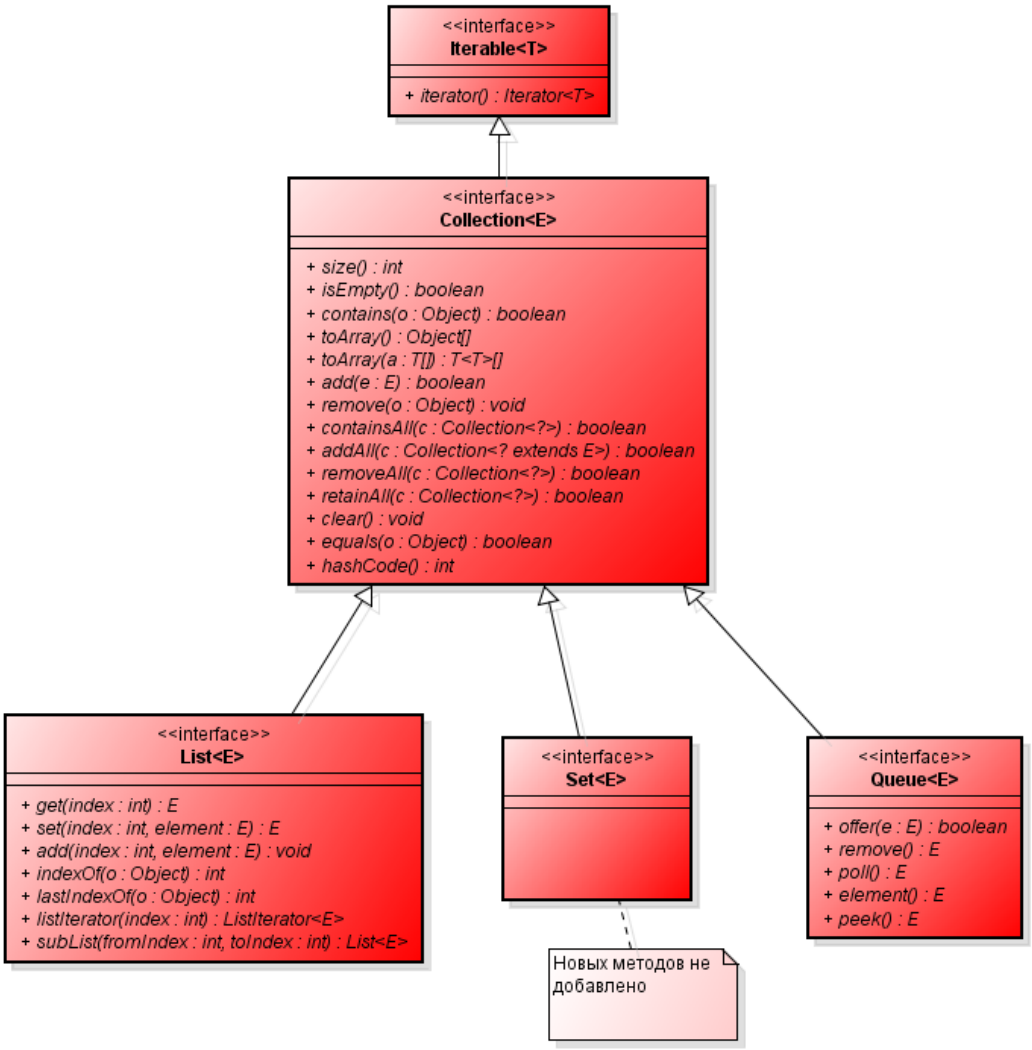
1. **Collection** - коллекция содержит набор объектов (элементов). Здесь определены основные методы для манипуляции с данными, такие как вставка (add, addAll), удаление (*remove, removeAll, clear*), поиск (*contains*)
2. **Map** - описывает коллекцию, состоящую из пар "ключ — значение". У каждого ключа только одно значение, что соответствует математическому понятию однозначной функции или отображения (тар). Такую коллекцию часто называют еще словарем (dictionary) или ассоциативным массивом (associative array). Никак НЕ относится к интерфейсу Collection и является самостоятельным.

Хотя фреймворк называется **Java Collections Framework**, интерфейс **map** и его реализации **входят** в фреймворк тоже!

Интерфейсы Collection и Map являются базовыми, но они не есть единственными. Их расширяют другие интерфейсы, добавляющие дополнительный функционал.

Интерфейс Collection

Рассмотрим основные интерфейсы, относящиеся к **Collection**:



Как видно с диаграммы, интерфейс **Collection** не является базовым. Интерфейс **Collection** расширяет интерфейс **Iterable**, у которого есть только

один метод iterator(). Это значит что любая коллекция, которая есть наследником **Iterable** должна возвращать итератор.

Итератор - объект, абстрагирующийся за единым интерфейсом доступ к элементам коллекции. Итератор это паттерн позволяющий получить доступ к элементам любой коллекции без вникания в суть ее реализации.

Как видим из рисунка, интерфейс **Collection** расширяют интерфейсы **List, Set** и **Queue**. Давайте рассмотрим, зачем нужен каждый.

1. **List** - Представляет собой неупорядоченную коллекцию, в которой допустимы дублирующие значения. Иногда их называют последовательностями (sequence). Элементы такой коллекции пронумерованы, начиная от нуля, к ним можно обратиться по индексу.
2. **Set** - описывает неупорядоченную коллекцию, не содержащую повторяющихся элементов. Это соответствует математическому понятию множества (set).
3. **Queue** - очередь. Это коллекция, предназначенная для хранения элементов в порядке, нужном для их обработки. В дополнение к базовым операциям интерфейса Collection, очередь предоставляет дополнительные операции вставки, получения и контроля.

Реализации интерфейса List

ArrayList - пожалуй самая часто используемая коллекция. ArrayList инкапсулирует в себе обычный массив, длина которого автоматически увеличивается при добавлении новых элементов.
Так как ArrayList использует массив, то время доступа к элементу по индексу минимально (В отличии от LinkedList). При удалении произвольного элемента из списка, все элементы находящиеся «правее» смещаются на одну ячейку влево, при этом реальный размер массива (его емкость, capacity) не

изменяется. Если при добавлении элемента, оказывается, что массив полностью заполнен, будет создан новый массив размером (n * 3) / 2 + 1, в него будут помещены все элементы из старого массива + новый, добавляемый элемент.

LinkedList - Двусвязный список. Это структура данных, состоящая из узлов, каждый из которых содержит как собственно данные, так и две ссылки («связки») на следующий и предыдущий узел списка. Доступ к произвольному элементу осуществляется за линейное время (но доступ к первому и последнему элементу списка всегда осуществляется за константное время — ссылки постоянно хранятся на первый и последний, так что добавление элемента в конец списка вовсе не значит, что придется перебирать весь список в поисках последнего элемента). В целом же, LinkedList в абсолютных величинах проигрывает ArrayList и по потребляемой памяти и по скорости выполнения операций.

Задание на лабораторную работу

Модифицировать приложение из предыдущей лабораторной работы, реализовав хранение данных таблицы с использованием библиотеки коллекций. Для этого реализовать класс RecIntegral, способный хранить одну запись таблицы. Для нечетных вариантов в качестве класса-коллекции выбрать ArrayList, для четных - LinkedList. Кроме того, добавить пару кнопок: очистить / заполнить, которые будут очищать таблицу и заполнять ее данными из коллекции соответственно. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Лабораторная работа №3

Обработка исключительных ситуаций

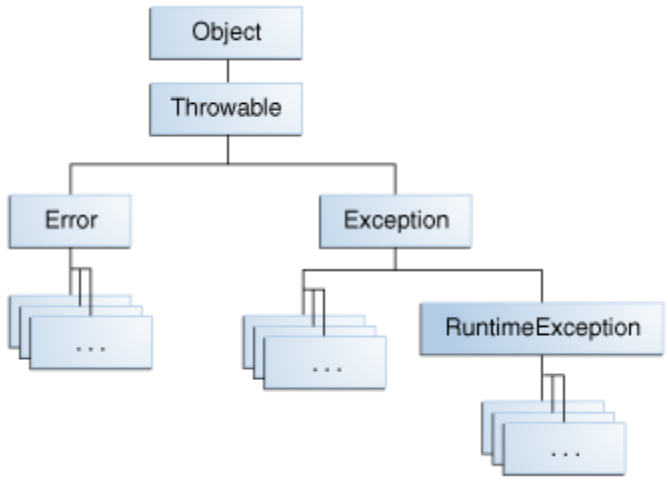
Цель работы: изучить механизм обработки исключительных ситуаций.

Теоретический материал

Исключениями или исключительными ситуациями (состояниями) называются ошибки, возникшие в программе во время её работы.

Все исключения в Java являются объектами. Поэтому они могут порождаться не только автоматически при возникновении исключительной ситуации, но и создаваться самим разработчиком.

Иерархия классов исключений:



Исключения делятся на несколько классов, но все они имеют общего предка — класс Throwable. Его потомками являются подклассы Exception и Error.

Исключения (Exceptions) являются результатом проблем в программе, которые в принципе решаемы и предсказуемы. Например, произошло деление на ноль в целых числах.

Ошибки (Errors) представляют собой более серьёзные проблемы, которые, согласно спецификации Java, не следует пытаться обрабатывать в собственной программе, поскольку они связаны с проблемами уровня JVM. Например, исключения такого рода возникают, если закончилась память, доступная виртуальной машине. Программа дополнительную память всё равно не сможет обеспечить для JVM.

В Java все исключения делятся на три типа: контролируемые исключения (checked) и неконтролируемые исключения (unchecked), к которым относятся ошибки (Errors) и исключения времени выполнения (RuntimeExceptions, потомок класса Exception).

Контролируемые исключения представляют собой ошибки, которые можно и нужно обрабатывать в программе, к этому типу относятся все потомки класса Exception (но не RuntimeException).

Обработка исключения может быть произведена с помощью операторов try…catch, либо передана внешней части программы. Например, метод может передавать возникшие в нём исключения выше по иерархии вызовов, сам его не обрабатывая.

Неконтролируемые исключения не требуют обязательной обработки, однако, при желании, можно обрабатывать исключения класса RuntimeException.

Задание на лабораторную работу

Модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса Exception, и генерировать исключение, если возникает попытка создать экземпляр класса RecIntegral со значениями, не являющимися числами в

диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводить диалог, содержащий предупреждение о некорректности введенных данных. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Лабораторная работа №4

Работа с файлами

Цель работы: изучить работу с файлами и механизмы сериализации данных.

Теоретический материал

Классы FileReader и FileWriter

Простейшие классы для чтения/записи из/в текстовый файл. Объекты этих классов создаются на основе имени файла в файловой системе. Например,

```
FileReader myfile = new FileReader("data.txt");

FileWriter myfile = new FileWriter("data.txt");
```

Оба класса имеют функцию getEncoding для получения текущей кодовой страницы (cp1251 - кодовая страница для русского Windows), а также конструкторы, которые позволяют задать нужную кодовую страницу в качестве второго параметра.

Имеют методы read и write для чтения/записи строки символов.

Интерфейс Serializable

Механизм сериализации основан на интерфейсе java.io.Serializable . Простое указание "implements Serializable" в объявлении класса и выполнение ряда простых требований позволяет сохранять/восстанавливать объекты этого класса стандартным образом.

При сохранении объектов применяется класс ObjectOutputStream , при восстановлении — ObjectInputStream .

Рассмотрим абстрактный пример. Пусть у нас в программе используется некоторый класс A и нам нужно обеспечить сохранение/восстановление объектов этого класса.

Тогда сам класс должен быть описан примерно так:

```
public class A implements Serializable {
      . . .
}
```

Сохранение объектов класса A в файле может быть реализовано следующим образом.

Пусть у нас есть объект класса A и на него ссылается переменная aObj , описанная как

```
A aObj;
```

Тогда следующий фрагмент обеспечит запись этого объекта в файл "A.ser".

```
ObjectOutputStream out = null;
try {
    out = new ObjectOutputStream(new BufferedOutputStream(
                                 new FileOutputStream("A.ser")));
    out.writeObject(aObj);
} catch ( IOException ex ) {
  ex.printStackTrace();
}
```

Для восстановления объекта из файла может быть использован следующий фрагмент:

```
ObjectInputStream in = null;
A restObj = null;
try {
   in = new ObjectInputStream(new BufferedInputStream(
                        new FileInputStream("A.ser")));
   restObj = (A)in.readObject();
} catch ( IOException ex ) {
  ex.printStackTrace();
}
```

Сам по себе интерфейс Serializable не имеет никаких методов и служит лишь для указания того, что мы собираемся сохранять и восстанавливать объекты, используя описанные выше возможности.

Задание на лабораторную работу

Модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла (JFileChooser).

Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Лабораторная работа №5

Многопоточность в Java

Цель работы: научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

Теоретический материал

В Java существует два основных способа организации многопоточных вычислений:

1. Создание своего дочернего класса на базе класса Thread. При необходимо переопределить метод run. Реализация этого метода будет работать в рамках отдельного потока.
2. Реализовать интерфейс Runnable. При этом в рамках класса необходимо определить метод run, который будет работать как отдельный поток.

Второй способ особенно удобен в тех случаях, когда класс должен быть унаследован от какого-либо другого класса и при этом вам нужна многопоточность. Так как в языке программирования Java нет множественного наследования, невозможно создать класс, для которого в качестве родительского будут выступать несколько классов. В этом случае реализация интерфейса Runnable является единственным способом решения задачи.

Задание на лабораторную работу

Модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта),

снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Варианты с номерами до 5 включительно реализуют многопоточность путем наследования от класса Thread, остальные реализуют интерфейс Runnable. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Лабораторная работа №6

Сетевое взаимодействие в Java

Цель работы: научиться создавать клиент-серверные приложения с использованием стандартных классов Java.

Теоретический материал

Java поддерживает два основных подхода к организации сетевого взаимодействия, которые отличаются используемыми сетевыми протоколами:

1. Протокол UDP. Не использует логическое соединение. Данные передаются дейтаграммами.
2. Протокол TCP. Использует логическое соединение и потоковую передачу данных.

Первый способ обладает большей скоростью передачи данных, но не обеспечивает подтверждение доставки передаваемых сообщений.

Задание на лабораторную работу

Модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру, обеспечивающую распределенное вычисление определенного интеграла на нескольких вычислительных узлах (клиентах) при этом каждый узел использует несколько нитей, как в предыдущей работе. Сервер не занимается вычислениями, а лишь реализует взаимодействие с пользователем и агрегацию результатов вычислений от клиентов. Нечетные варианты используют протокол UDP, а четные TCP. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Приложение 1. Варианты лабораторных заданий

Номер варианта	Функция
1	$\frac{1}{x}$
2	$\sin(x)$
3	$\cos(x)$
4	$tg(x)$
5	e^{-x}
6	$\sqrt{x}$
7	$\frac{1}{\ln(x)}$
8	$\sin(x^2)$
9	$\cos(x^2)$
10	$\frac{e^x}{x}$

Приложение 2. Требования к оформлению лабораторных работ

По каждой лабораторной работе необходимо составить отчет, который должен содержать:

- титульный лист;
- название и цель работы;
- лабораторное задание в соответствии с вариантом;
- ход выполнения лабораторной работы, включающий словесное описание алгоритма работы программы или его графическое представление;
- листинг программы;
- результаты выполнения программы для различных случаев;
- выводы.

Отчет должен предоставляться в печатном виде в соответствии с представленными выше требованиями.